Information System of Research-Oriented University

OOP Final Project — Console Application in Java

Team: 3 members

Team leader: Alibek Andanuarbek

Member 2: _____

Member 3: _____

Course: Object-Oriented Programming, 2025/26

Date: April 2026

Repository: github.com/alibekn6/oop-final

Contents

1. Introduction
2. Architecture (Use Case & Class Diagrams)
3. Class Hierarchy
4. Design Patterns
5. Main Flows
    1. Course Registration
    2. Putting Marks
    3. Research
6. Custom Exceptions
7. Serialization
8. Project Management — Problems & Solutions
9. Team Communication
10. Conclusion

1. Introduction

This project is a console-based information system for a research-oriented university, written in Java. It models the people, courses, marks, and research activity of a university and exposes their interactions through a single demo class. The project is the final deliverable of the Object-Oriented Programming course and is built around a strict scope laid out in the team's technical specification: 12 mandatory classes, four mandatory user-facing flows (authentication, course registration, marking, research), serialization of all state, and at least four design patterns.

Production-ready behaviour was an explicit non-goal. The brief is to demonstrate object-oriented thinking — abstract types and interfaces, encapsulation, controlled use of inheritance, composition where inheritance does not fit, custom checked exceptions, the Java Collections framework, and serialization. Where the spec gave us a choice (for example, "Researcher could be an interface, an abstract class, or a Decorator — figure it out"), we made the pragmatic call and document it in this report.

The system has 26 source files arranged in seven packages under `com.uni`. A single class, `UniversityDemo`, runs every required scenario end-to-end and prints results to the console — including the failure paths that throw our four custom exceptions.

2. Architecture

2.1 Package layout

```
src/com/uni/
├── enums/         8 enums  (Language, LessonType, TeacherTitle, ...)
├── exceptions/    4 custom exceptions
├── models/        14 classes  (User hierarchy, academic, research)
├── comparators/   3 Comparator<ResearchPaper> strategies
├── factory/       UserFactory  (Factory pattern)
├── storage/       DataStore    (Singleton + persistence)
└── UniversityDemo.java
```

The packaging is layered: `enums` and `exceptions` have no internal dependencies; `models` depends only on `enums` and `exceptions`; `comparators`, `factory`, and `storage` depend on `models`. This keeps the coupling acyclic and easy to reason about.

2.2 Use Case diagram

Actors: Admin, Manager, Teacher, Student, and Researcher (a role any User can take on). The full diagram, drawn in StarUML, is exported to `diagrams/use-case.png`.

[Insert use-case diagram image here]

2.3 Class diagram

The class diagram, also drawn in StarUML, is exported to `diagrams/class-diagram.png`. The source file is `classdiOOPFINAL.mdj` and was kept in sync with the code via reverse engineering.

[Insert class diagram image here]

3. Class Hierarchy

The user inheritance tree:

```
User (abstract, Serializable)
├── Employee (abstract)
│   ├── Teacher              implements Researcher
│   ├── Manager
│   ├── Admin
│   └── ResearcherEmployee   implements Researcher
└── Student                  implements Researcher

Researcher (interface, extends Serializable)
└── ResearcherDecorator implements Researcher  (wraps any User)
```

3.1 User (abstract)

The base of every person in the system. It owns the credentials and name fields and exposes a plain `login(login, password)` method used by the authentication flow.

```
public abstract class User implements Serializable {
    protected long id;
    protected String login;
    protected String password;
    protected String firstName;
    protected String lastName;
    protected Language language;

    public boolean login(String login, String password) {
        return Objects.equals(this.login, login)
                && Objects.equals(this.password, password);
    }
}
```

3.2 Employee and its concrete subclasses

`Employee` adds salary, hire date, and an inbox of `Messages`. Its concrete subclasses are `Teacher`, `Manager`, `Admin`, and `ResearcherEmployee`.

Class	Responsibilities
Teacher	Owns active courses; can put marks (3 attestations); rates accumulate; if PROFESSOR — automatically a Researcher.
Manager	Adds courses; assigns instructors; approves student registrations; produces statistical reports; promotes any User to Researcher via Decorator.
Admin	Adds, removes, and updates users; reads audit logs.
ResearcherEmployee	Employee whose only role is research. Always a Researcher.

3.3 Student

Bachelor student (years 1..4). Holds a transcript (`Map<Course, Mark>`), credit and fail counters, an optional `Researcher` supervisor (only meaningful at year 4), and an optional researcher role gated by a flag. All three invariants — credit cap, fail cap, supervisor h-index — are enforced by checked exceptions.

```
public class Student extends User implements Researcher {
    public static final int MAX_CREDITS = 21;
    public static final int MAX_FAILS   = 3;
    private final Map<Course, Mark> transcript = new LinkedHashMap<>();
    private Researcher supervisor;
    ...
}
```

3.4 Researcher (interface)

Anyone capable of doing research. Extends `Serializable` so any list of researchers can be saved.

```
public interface Researcher extends Serializable {
    double calculateHIndex();
    void   printPapers(Comparator<ResearchPaper> comparator);
    List<ResearchPaper> getPapers();
    void   publishPaper(ResearchPaper paper);
    String getResearcherName();
    default boolean isResearcher() { return true; }
}
```

3.5 Course / Mark / Lesson

`Course` implements `Comparable<Course>` and supports multiple lecture and practice instructors. `Mark` stores three attestation scores (max 30/30/40), computes the total and a letter grade, and exposes `isPassed()`. `Lesson` couples a course with a type (LECTURE/PRACTICE), room, and schedule.

3.6 Research models

`ResearchPaper` is `Comparable<ResearchPaper>` (newest first by default) and exposes `getCitation(CitationFormat)` for both PLAIN_TEXT and BibTeX. `ResearchProject` validates new participants and rejects non-researchers.

3.7 Why Researcher is an interface, not a class

The brief deliberately leaves this open. We chose *interface plus optional Decorator*:

- **Interface**, because being a researcher is a role that crosses the regular User hierarchy: a Teacher, a Student, or a plain Employee can all hold the role. Forcing them to share a common abstract *class* would have broken the User tree.
- **Decorator** (`ResearcherDecorator`), because the spec asks the Manager to be able to grant the role to *any* User at runtime without recompiling them. Wrapping with a decorator gives that without modifying the wrapped class.

4. Design Patterns

4.1 Singleton — DataStore

The system has a single source of truth for state, so only one `DataStore` instance exists. The constructor is private; a synchronised `getInstance()` hands it out.

```
public class DataStore implements Serializable {
    private static DataStore instance;
    private DataStore() {}
    public static synchronized DataStore getInstance() {
        if (instance == null) instance = new DataStore();
        return instance;
    }
}
```

4.2 Factory — UserFactory

Creating five different User subclasses through one entry point keeps the demo and admin-add-user paths simple. The `UserType` enum decides which subclass to instantiate.

```
public static User createUser(UserType type, long id,
                              String login, String password,
                              String firstName, String lastName,
                              String email, Language language,
                              Object... extra) {
    switch (type) {
        case STUDENT:             return new Student(...);
        case TEACHER:             return new Teacher(...);
        case MANAGER:             return new Manager(...);
        case ADMIN:               return new Admin(...);
        case RESEARCHER_EMPLOYEE: return new ResearcherEmployee(...);
    }
}
```

4.3 Decorator — ResearcherDecorator

Lets the Manager promote any User into a Researcher at runtime without changing that user's class.

```
public class ResearcherDecorator implements Researcher, Serializable {
    private final User originalUser;
    private final List<ResearchPaper> papers = new ArrayList<>();
    public ResearcherDecorator(User user) { this.originalUser = user; }
    public void publishPaper(ResearchPaper p) { papers.add(p); }
    ...
}

// Manager.makeResearcher
Researcher r = new ResearcherDecorator(someUser);
DataStore.getInstance().addResearcher(r);
```

4.4 Strategy — Paper Comparators

`Researcher.printPapers(Comparator c)` takes a sorting strategy at runtime. Three concrete strategies are provided.

```
researcher.printPapers(new DateComparator());        // newest first
researcher.printPapers(new CitationsComparator());   // most cited first
researcher.printPapers(new PaperLengthComparator()); // longest first
```

5. Main Flows

5.1 Course Registration

The flow is split between the student (who initiates) and the Manager (who finalises). Three invariants are enforced:

- Total credits ≤ `MAX_CREDITS` (21) — else `CreditLimitException`.
- Fail count < `MAX_FAILS` (3) — else `MaxFailedReachedException`.
- Course target year matches student's year (warning only).

```
// 1) student initiates
student.registerForCourse(course);
// 2) manager finalises
manager.approveRegistration(student, course);
// → adds student to course.students,
//   creates a fresh Mark(0, 0, 0) in transcript,
//   adds course.credits to currentCredits
```

5.2 Putting Marks

A Teacher writes one of three attestation scores into the existing Mark; on a failed final the Student's fail counter increments and can throw `MaxFailedReachedException`.

```
teacher.putMark(student, course, 1, 25);   // 1st att, max 30
teacher.putMark(student, course, 2, 22);   // 2nd att, max 30
teacher.putMark(student, course, 3, 30);   // final,   max 40
```

Letter grade is derived from the total:

Total	Letter
≥ 90	A
≥ 75	B
≥ 60	C
≥ 50	D
< 50	F (failed)

5.3 Research

The research flow combines four operations:

- **Publish:** `researcher.publishPaper(paper)`
- **Sort & print:** `researcher.printPapers(comparator)` — Strategy.
- **H-index:** standard definition — largest h such that h papers have ≥ h citations each.
- **Top cited researcher:** `DataStore.printTopCitedResearcher()` sums citations across each researcher's papers and picks the maximum.

```
static double hIndex(List<ResearchPaper> papers) {
    List<ResearchPaper> sorted = new ArrayList<>(papers);
    sorted.sort((a, b) -> Integer.compare(b.getCitations(), a.getCitations()));
    int h = 0;
    for (int i = 0; i < sorted.size(); i++) {
        if (sorted.get(i).getCitations() >= i + 1) h = i + 1;
        else break;
    }
    return h;
}
```

6. Custom Exceptions

All four are checked (`extends Exception`) — they represent business rule violations the caller must handle.

Exception	Thrown when	Thrown from
LowHIndexException	Supervisor's h-index < 3	Student.setSupervisor
NotAResearcherException	Non-researcher tries to join a project	ResearchProject.addParticipant
CreditLimitException	Student would exceed 21 credits	Student.registerForCourse
MaxFailedReachedException	Student fails for the 3rd time, or tries to register with 3 fails	Student.registerForCourse, Student.recordFail, Teacher.putMark

The demo deliberately exercises every one of these.

7. Serialization

Persistence is handled by `DataStore.save()` and `DataStore.load()`. Java's built-in object serialization is used: every persistent class implements `Serializable`, including the `Researcher` interface and the three `Comparator`s. The default file is `data.ser`.

```
public void save(String path) {
    try (ObjectOutputStream oos =
             new ObjectOutputStream(new FileOutputStream(path))) {
        oos.writeObject(this);
    } catch (IOException e) { ... }
}

public static DataStore load(String path) {
    try (ObjectInputStream ois =
             new ObjectInputStream(new FileInputStream(path))) {
        instance = (DataStore) ois.readObject();
        return instance;
    } catch (IOException | ClassNotFoundException e) { ... }
}
```

The demo verifies a complete round-trip: it saves, calls `DataStore.resetForTesting()` (wiping the singleton in memory), then `load()`s and re-prints a transcript. All object graph relations — students ↔ marks ↔ courses ↔ teachers — survive intact.

8. Project Management — Problems and Solutions

8.1 Splitting work between three people

The team divided the codebase along three vertical slices that share the same User abstract base but otherwise touch disjoint files:

- **Member 1 (team lead):** User hierarchy, Employee subclasses, `DataStore`, `UserFactory`, all enums, authentication.
- **Member 2:** Student, Course, Mark, Lesson, the registration and marking flows, two of the custom exceptions.
- **Member 3:** Researcher interface, ResearcherDecorator, ResearchPaper, ResearchProject, the three paper comparators, and the two research-related exceptions.

8.2 Concrete problems we hit

Naming clash on `getTranscript()`. The spec listed both `getTranscript(): String` (printable report) and a transcript field of type `Map<Course, Mark>` with implied accessor. Java cannot have two methods of the same name returning different types, so the map accessor was renamed to `getTranscriptMap()` and all internal callers (Course, Manager, Teacher) were updated.

How to model Researcher. The spec was deliberately ambiguous. We rejected an abstract class because it would have collided with the Employee/User hierarchy — a Student cannot also extend ResearcherBase. We chose an interface plus a Decorator (see §3.7) so the role can be both compile-time (Teacher/Student/ResearcherEmployee implementing it directly) and runtime (Manager wrapping any User).

Spurious lint warnings on `List<User>` and similar. Java's `-Xlint:serial` reports a "non-transient instance field declared with a non-serializable type" even when the parameter type implements Serializable, because the static type is an abstract class. We made the `Researcher` interface extend `Serializable` to silence that subset; the rest are benign and do not affect runtime behaviour.

De-duplicating researchers in `DataStore`. A user can be both a Teacher (Researcher by interface) and wrapped by a ResearcherDecorator. Listing both would double-count their papers. `getAllResearchers()` deduplicates by tracking the underlying User of each decorator and skipping the user's own direct entry when they are wrapped.

8.3 Git workflow

All work was done on `main` with frequent rebases. Commits were grouped by logical layer rather than by file count: a full snapshot of the code reaches the repository in seven commits (project setup → enums + exceptions → User hierarchy → academic + infra models → research models → factory + DataStore → demo).

9. Team Communication

Day-to-day coordination took place in a Telegram group. Screenshots are attached below.

[Telegram chat screenshot 1]

[Telegram chat screenshot 2]

10. Conclusion

The system meets every requirement of the technical specification: all 12 mandatory classes, the four required flows (authentication, course registration, marking, research), four design patterns (Singleton, Factory, Decorator, Strategy), four custom exceptions, full Java serialization, and a demo class that walks every scenario including the failure paths.

From a learning perspective, the most useful lessons were *where not to inherit* (Researcher across the User tree), the difference between compile-time and runtime polymorphism (interface vs. Decorator), and the cost of getting equals, hashCode, and Comparable consistent with serialization round-trips.

What did not make it into scope: schedule generation, attendance, regex-based search, recommendation letters, and a real multi-language UI. They are deliberately listed in the technical spec as “time permitting” items.

Source code, demo output, and this report are available in the project repository.